

[Dashboard](#) / [My courses](#) / [ITB_IF2010_2_2526](#) / [UAS](#) / [POST UAS](#)

Started on	Sunday, 21 June 2026, 11:51 PM
State	Finished
Completed on	Sunday, 21 June 2026, 11:59 PM
Time taken	7 mins 33 secs
Grade	400.00 out of 400.00 (100%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Chess Movement

Inheritance, Interface, Polymorphism, & 4-Sekawan.

Anda bertugas untuk mengimplementasikan pergerakan beberapa bidak catur (Bishop, Knight, Rook).

Implementasi diharapkan memanfaatkan konsep OOP berupa **Inheritance, Interface, Polymorphism, & 4-Sekawan**.

Task:

No	Task Desc
1	Membuat Interface Aksi yang berisi 8 method ("atas", "atasKanan", "atasKiri", "kanan", "kiri", "bawahKanan", "bawahKiri", "bawah"). Semua method ini memiliki default behaviour sprint "[X] Gerakan tidak valid untuk bidak ini!\n" dan berparameter "Optional<Integer> steps". Ini merupakan tindakan preventif jika kelas yang megimplements interface ini tidak memiliki aksi valid untuk method terpilih.
2	Lengkapi implementasi kelas Bidak, Bishop, Knight, Rook agar program berjalan sesuai expected behaviour. Simbol Bishop = 'B', Knight = 'K', Rook = 'R'

Expected Alur program

- Program akan inisiasi 3 bidak (Bishop, Knight, Rook) di posisi tertentu
- Program akan menampilkan posisi setiap bidak dengan melakukan print bidak (cetakPapan)
- User diminta untuk memilih id bidak yang ingin dimainkan, jika 0 program akan berhenti
- User diminta untuk memilih arah pergerakan
- Jika bidak yang dipilih (Bishop/Rook), user akan diminta jumlah langkah dari arah pergerakan
- Program akan mengeksekusi fungsi terkait sehingga posisi berubah. Jika terjadi kondisi tidak valid (cetak *error*, aksi dibatalkan, posisi di-rollback).
- Program akan *looping* mulai dari langkah ke-2, hingga *input* user [0]

Additional Notes

- [ChessMovement.zip](#)
- [Driver.java](#)
- [Petak.java](#)
- Kumpulkan Aksi.java, Bidak.java, Bishop.java, Knight.java, Rook.java ke dalam ChessMovement.zip
- Anda mungkin akan membutuhkan Java.Util.Optional, silakan buka dokumentasi penggunaannya [di sini](#)

Java 8

 [ChessMovement.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.19 sec, 28.36 MB
2	10	Accepted	0.16 sec, 29.61 MB
3	10	Accepted	0.17 sec, 26.87 MB
4	10	Accepted	0.13 sec, 30.89 MB
5	10	Accepted	0.14 sec, 28.80 MB
6	10	Accepted	0.16 sec, 28.51 MB
7	10	Accepted	0.15 sec, 30.10 MB
8	10	Accepted	0.15 sec, 28.85 MB
9	10	Accepted	0.15 sec, 28.77 MB
10	10	Accepted	0.27 sec, 27.14 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Kamu diminta untuk mengimplementasikan sebuah kelas generik **Storage<T>** yang mensimulasikan penyimpanan berkapasitas terbatas. Sistem ini akan menyimpan data yang direpresentasikan oleh objek dengan tipe T dengan menggunakan **String** sebagai kunci.

Penyimpanan ini harus mendukung operasi penambahan, pengambilan, penghapusan, dan melihat seluruh data. Kelas juga harus dapat menangani berbagai kondisi tidak terduga melalui Exceptions.

Lengkapi **TODO** yang ada pada file [Storage.java](#). Anda diberikan kerangka kode serta [Main.java](#) untuk membantu Anda melakukan pengujian secara mandiri. [File Exception](#) sudah disediakan. **Kumpulkan kembali Storage.java.**

[Output Main](#)

Java 8

 [Storage.java](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.21 sec, 35.90 MB
2	20	Accepted	0.09 sec, 28.14 MB
3	20	Accepted	0.23 sec, 33.55 MB
4	20	Accepted	0.12 sec, 28.43 MB
5	20	Accepted	0.11 sec, 28.99 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Seorang insinyur sistem sedang membangun **pipeline analitik** untuk jaringan sensor industri. Setiap sensor menghasilkan serangkaian pembacaan numerik, beberapa sensor menghasilkan nilai integer (misalnya jumlah getaran), sementara yang lain menghasilkan nilai desimal (misalnya suhu dalam Celsius).

Data dari banyak sensor dikumpulkan ke dalam *batch*. Setiap *batch* memiliki sebuah label dan sejumlah nilai pembacaan. Karena jumlah batch bisa sangat banyak, pemrosesan dilakukan secara paralel. Beberapa *thread* mengerjakan bagian berbeda dari data secara bersamaan, lalu hasilnya digabungkan.

File yang Tersedia

Jangan diubah:

- **ThreadTracker.java** — utilitas untuk melacak dan memverifikasi thread yang benar-benar dijalankan. Pelajari method-method yang tersedia di dalamnya dan gunakan dengan benar di implementasimu.

Perlu dilengkapi:

- **DataBatch.java** — skeleton kelas generik untuk satu batch data. Struktur dan nama method sudah tersedia, namun *type parameter* dan implementasinya belum diisi.
- **BatchProcessor.java** — skeleton kelas yang mengimplementasikan **Runnable**. Struktur sudah tersedia, namun tipe generik pada parameter dan implementasinya belum diisi.
- **DataUtils.java** — skeleton kelas utilitas dengan dua method statis. Struktur sudah tersedia, namun tipe generik pada parameter dan implementasinya belum diisi.
- **Main.java** — driver program yang membaca input, membagi batch ke thread, dan mencetak output. Isi blok **// TODO** yang ada di dalamnya; bagian lain jangan diubah.

Skeleton yang diberikan menggunakan *raw type* sebagai placeholder. Tugasmu adalah menentukan *type parameter* dan *wildcard* yang tepat, lalu mengisi implementasinya. Detail spesifikasi setiap method sudah tersedia sebagai komentar di dalam masing-masing skeleton. Gunakan **Main.java** sebagai acuan untuk mengetahui tipe apa yang diharapkan.

Format Input

```
N T
TYPE label k v1 v2 ... vk
TYPE label k v1 v2 ... vk
...
(N baris)
```

- **N** — jumlah batch
- **T** — jumlah thread (dijamin $T \leq N$)
- **TYPE** — tipe batch: **INT** atau **DOUBLE**
- **label** — nama batch (string tanpa spasi)
- **k** — jumlah nilai dalam batch
- **v1 ... vk** — nilai-nilai batch sesuai tipe

Format Output

Batch dibagi ke thread secara merata. Jika **N** tidak habis dibagi **T**, thread dengan indeks lebih kecil mendapat satu batch ekstra.

```
Thread 0:
Batch <label>: sum = <sum>
...

Thread 1:
Batch <label>: sum = <sum>
...

(blank line setelah setiap blok thread)

Total items processed: <total_item>
Total sum: <total_sum>
Max: <max_item>
```

Semua nilai desimal dicetak dengan satu angka di belakang titik (**%1f**). **Total items processed** adalah jumlah elemen (bukan batch) yang diproses oleh semua thread.

Contoh Input dan Output

Input

Output

Penjelasan:

- $N=5, T=2 \rightarrow \text{base}=2, \text{extra}=1$
- Thread 0 mendapat 3 batch: *sensor_a* (sum=6.0), *sensor_b* (sum=100.0), *sensor_c* (sum=4.0)
- Thread 1 mendapat 2 batch: *sensor_d* (sum=18.0), *sensor_e* (sum=300.0)
- Total item: $3+4+2+3+2 = 14$
- Total sum: $6+100+4+18+300 = 428.0$
- Max item: 200 (dari *sensor_e*)

Batasan

- $1 \leq T \leq N \leq 100$
- $1 \leq k \leq 1.000$ (jumlah item per batch)
- Label batch hanya mengandung huruf, angka, dan underscore
- Nilai INT dalam rentang int; nilai DOUBLE dalam rentang double

Catatan Teknis

- Dilarang menggunakan **Thread.sleep()** untuk sinkronisasi.
- Gunakan **Locale.US** di semua pemanggilan **String.format** dan **printf** agar format desimal menggunakan titik (bukan koma).

Format Pengumpulan

Kumpulkan zip yang berisi: **DataBatch.java**, **BatchProcessor.java**, **DataUtils.java**, dan **Main.java**.

Java 8

 [ans3.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	14	Accepted	0.17 sec, 34.16 MB
2	14	Accepted	0.18 sec, 34.33 MB
3	14	Accepted	0.18 sec, 34.52 MB
4	14	Accepted	0.17 sec, 33.80 MB
5	14	Accepted	0.18 sec, 36.35 MB
6	14	Accepted	0.18 sec, 36.45 MB
7	16	Accepted	0.18 sec, 33.87 MB

Question **4**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Smart Home Controller

Setelah Dr. Neroifa dibantu oleh Kebin dan Stewart terkait pengelolaan kedai jus dan usaha Vending Machine nya, Dr. Neroifa akhirnya bisa menabung untuk membeli rumah. Satu tahun kemudian, Dr. Neroifa akhirnya berhasil membeli rumah impiannya. Namun, Dr. Neroifa ingin melakukan upgrade rumahnya menjadi Smart Home beserta perabotan-perabotan di rumahnya bisa ia kontrol melalui aplikasi. Anda diminta untuk membantu Dr. Neroifa agar sistem kontrol smart homenya dapat berjalan dengan baik dan **bisa diterapkan juga jika ia menambah perabotan baru ke depannya**.

Spesifikasi Kelas

Nama Kelas	Spesifikasi
------------	-------------

Nama Kelas	Spesifikasi
SmartHomeController	• SmartHomeController merupakan utility class yang menyediakan operasi untuk memeriksa status dan menjalankan perintah pada berbagai jenis perangkat smart home saat program sedang berjalan. Kelas ini dirancang agar dapat bekerja dengan objek perangkat apa pun tanpa bergantung secara langsung pada kelas seperti SmartLamp atau SmartFan. Seluruh informasi mengenai atribut dan method perangkat harus diperoleh menggunakan Java Reflection. • Atribut: tidak ada. • Konstruktor: Kelas ini tidak boleh dapat diinstansiasi dari luar kelas karena hanya berisi operasi statis. • Method: ◦ <code>static void printStatus(Object device)</code> mencetak seluruh atribut yang dideklarasikan pada class dari objek <code>device</code>, termasuk atribut yang memiliki akses <code>private</code>. Setiap atribut dicetak menggunakan format: <pre>namaAtribut = nilaiAtribut</pre> Contoh: <pre>name = Lampu Kamar brightness = 80 active = false</pre> Jika suatu atribut tidak dapat diakses, method mencetak: <pre>FIELD_ACCESS_ERROR</pre> ◦ <code>static void printCommands(Object device)</code> mencetak nama seluruh method yang dideklarasikan pada class dari objek <code>device</code>, termasuk method dengan akses <code>private</code>. Nama-nama method harus diurutkan secara ascending berdasarkan urutan alfabet agar output konsisten. Setiap nama method dicetak pada baris yang berbeda. Contoh: <pre>increaseBrightness resetBrightness turnOff turnOn</pre> ◦ <code>static void execute(Object device, String command)</code> mencari dan menjalankan method tanpa parameter pada objek <code>device</code> yang memiliki nama sama dengan nilai <code>command</code>. Method yang dijalankan dapat memiliki akses <code>public</code> maupun <code>private</code>. Jika method berhasil ditemukan, method tersebut dijalankan pada objek <code>device</code>. Jika method dengan nama yang sesuai tidak ditemukan, cetak: <pre>COMMAND_NOT_FOUND</pre> Jika method ditemukan tetapi tidak dapat diakses atau mengalami kegagalan ketika dijalankan, cetak: <pre>COMMAND_EXECUTION_ERROR</pre> • Seluruh operasi pada SmartHomeController harus bersifat umum dan dapat digunakan terhadap class perangkat lain yang tidak disebutkan secara langsung pada soal. Oleh karena itu, implementasi tidak boleh bergantung pada pengecekan tipe atau pemanggilan method khusus terhadap SmartLamp, SmartFan, maupun class perangkat tertentu lainnya.

Untuk memudahkan pengujian class, berikut program **Main** yang dapat Anda coba beserta file-file lainnya yang terkait

Format Pengumpulan

Kumpulkan file: **SmartHomeController.java**.
Pastikan program dapat dikompilasi dan dijalankan dengan benar.

Java 8

 [SmartHomeController.java](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	5	Accepted	0.16 sec, 33.30 MB
2	5	Accepted	0.16 sec, 33.94 MB
3	5	Accepted	0.16 sec, 34.16 MB
4	5	Accepted	0.16 sec, 33.77 MB
5	5	Accepted	0.17 sec, 35.36 MB
6	5	Accepted	0.15 sec, 31.96 MB
7	5	Accepted	0.17 sec, 33.79 MB
8	5	Accepted	0.16 sec, 33.88 MB
9	5	Accepted	0.16 sec, 33.63 MB
10	5	Accepted	0.15 sec, 33.30 MB
11	5	Accepted	0.16 sec, 33.88 MB
12	5	Accepted	0.15 sec, 34.04 MB
13	5	Accepted	0.16 sec, 33.91 MB
14	5	Accepted	0.16 sec, 32.67 MB
15	5	Accepted	0.17 sec, 33.79 MB
16	5	Accepted	0.16 sec, 35.78 MB
17	5	Accepted	0.15 sec, 34.06 MB
18	5	Accepted	0.16 sec, 33.42 MB
19	5	Accepted	0.15 sec, 34.12 MB
20	5	Accepted	0.16 sec, 33.86 MB

[← UAS](#)

Jump to...